Computer Architecture and System Software

Lecture 06: Midterm Results, Assembly Language

Instructor:
David Bosc
Applied Computer Science
University of Winnipeg

Announcements

- Hand back midterms after break, go over solutions after lecture content
  - If you did not attend lecture, please see Connie Arnhold in 3D13 to pick it up
- Assignment 3 will be available this Friday
  - Due Friday, May 13th
  - All Assembly coding

Debugging

Using the debugger

- It is possible examine the contents of memory and the CPU registers after the execution of each instruction

- In DOSBOX, type

  debug <filename>.exe

- Type ? to see available commands

Debugging

- Useful commands for us:
  - T (trace [num]) – execute *num* instructions and view CPU registers and flags before the instruction starts
  - P (proceed [num]) – same as trace but skips through loops and interrupt calls
  - D (dump [num]) – view contents of data segment from address DS:*num*
    - *Can also specify other segments (SS, CS, ES)*
  - U (unassemble [num]) – View the assembly code instructions in the code segment with offset address *num*
  - A (assemble [num]) – You will be prompted for an instruction to be executed at address *num*

Tracing

- First few instructions are setting up your data and code segments
- Contents of registers are shown **before** instruction executes
- You still have input/output functionality in trace mode
- Also visible:
  - Address of current instruction, address of code segment, machine code equivalent instruction (hex form), flags
- Variable name will not be displayed – only their memory addresses

Tracing


```
C:\>debug COMPARE.EXE
-t 4

AX=076A  BX=0000  CX=0124  DX=0000  SP=0064  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=0770  CS=0777  IP=0003   NV UP EI PL NZ NA PO NC
0777:0003 8ED8          MOV     DS,AX

AX=076A  BX=0000  CX=0124  DX=0000  SP=0064  BP=0000  SI=0000  DI=0000
DS=076A  ES=075A  SS=0770  CS=0777  IP=0005   NV UP EI PL NZ NA PO NC
0777:0005 B87007        MOV     AX,0770

AX=0770  BX=0000  CX=0124  DX=0000  SP=0064  BP=0000  SI=0000  DI=0000
DS=076A  ES=075A  SS=0770  CS=0777  IP=0008   NV UP EI PL NZ NA PO NC
0777:0008 8ED0          MOV     SS,AX

AX=0770  BX=0000  CX=0124  DX=0000  SP=0064  BP=0000  SI=0000  DI=0000
DS=076A  ES=075A  SS=0770  CS=0777  IP=000E   NV UP EI PL NZ NA PO NC
0777:000E B40A          MOV     AH,0A
-_
```

Flags register

- 8 status flags displayed in debugger
  - Overflow, Direction, Interrupt, Sign, Zero, Auxillary, Parity, and Carry
  - 1's and 0's are displayed as 2 character abbeviations
- OV (OVerflow) NV [No oVerflow]
  DN (decrement) UP (increment)
  EI (enabled) DI (disabled)
  NG (negative) PL (positive)
  ZR [zero] NZ [Not zero]
  AC [Auxiliary Carry] NA [No AC]
  PE (even) PO (odd)
  CY [Carry] NC [No Carry]

Memory dump

- View contents of data segment, or specific range of memory addresses
- Important to trace through first few instructions of program first to initialize your data segment first
- In the next image we'll see:
  - Left: Memory addresses
  - Middle: Contents (hex)
  - Right: Contents decoded using ASCII

Memory dump


```
C:\>debug COMPARE.EXE
-d 0
075A:0000  CD 20 FF 9F 00 EA FF FF-AD DE 4F 03 A3 01 8A 03   . ........O.....
075A:0010  A3 01 17 03 A3 01 92 01-01 01 01 00 02 FF FF FF   ................
075A:0020  FF FF FF FF FF FF FF FF-FF FF FF FF 50 07 4C 01   ............P.L.
075A:0030  63 06 14 00 18 00 5A 07-FF FF FF FF 00 00 00 00   c.....Z.........
075A:0040  05 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ................
075A:0050  CD 21 CB 00 00 00 00 00-00 00 00 00 00 00 00 00   .!..............
075A:0060  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ................
075A:0070  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ................
-
```

Unassemble

- View instructions in the code segment starting at specified offset address for any IA32 .exe file

- In the next image we'll see:
  - Left: Address of instruction + machine code
    - Ex: 0777:0000 B86A07
  - Right: Assembly code instruction equivalent
    - Ex: MOV AX, 076A

Unassemble


```
C:\>debug COMPARE.EXE
-u 0
0777:0000 B86A07        MOV     AX,076A
0777:0003 8ED8          MOV     DS,AX
0777:0005 B87007        MOV     AX,0770
0777:0008 8ED0          MOV     SS,AX
0777:000A 8D160000      LEA     DX,[0000]
0777:000E B40A          MOV     AH,0A
0777:0010 B40A          MOV     AH,0A
0777:0012 CD21          INT     21
0777:0014 B20A          MOV     DL,0A
0777:0016 B402          MOV     AH,02
0777:0018 CD21          INT     21
0777:001A 8D360000      LEA     SI,[0000]
0777:001E 83C602        ADD     SI,+02
-_
```

Assemble

- Execute an instruction at address specified by user
- In the image, we unassembled the code segment from address 0 to view the addresses of each instruction currently in the code segment
- Next, we assembled a new instruction at address E (or 000E) by typing 'a E'
- The old instruction at 0777:000E was MOV AH, 0A. We replaced it with MOV AH, 09.
- Tracing through the program normally would now run our new instruction instead.

Assemble


```
C:\>debug COMPARE.EXE
-u 0
0777:0000 B86A07        MOV     AX,076A
0777:0003 8ED8          MOV     DS,AX
0777:0005 B87007        MOV     AX,0770
0777:0008 8ED0          MOV     SS,AX
0777:000A 8D160000      LEA     DX,[0000]
0777:000E B40A          MOV     AH,0A
0777:0010 B40A          MOV     AH,0A
0777:0012 CD21          INT     21
0777:0014 B20A          MOV     DL,0A
0777:0016 B402          MOV     AH,02
0777:0018 CD21          INT     21
0777:001A 8D360000      LEA     SI,[0000]
0777:001E 83C602        ADD     SI,+02
-a E
0777:000E mov AH, 9
0777:0010
-       q

C:\>
```

Assemble

- Notice that the size of instructions varies.
- LEA DX, [0000] is a 4-byte large instruction. The machine code equivalent is 0x8D160000.
- MOV AH, 0A is a 2-byte instruction. The machine code equivalent is 0xB40A.

- Assembling using the debugger must be done with caution to conserve instruction size.

Debugging

- To stop assembling, simply press enter
- To quit the debugger, type q

Branching and Loops

Branching and Loops

- Normally, CPU executes instructions one after another, in order they are written
- Branching and loops require different behaviour
- Need to be able to ask questions about our code
  - Is my result bigger than…
  - Is my result positive or negative?
  - Etc.

- After any mathematical operation, the flags register tracks various properties of the result

Recall: Flag Register

- Overflow flag: Set if a signed overflow occurs
- Direction flag: Used to indicate direction of processing for string manipulations (0 = forward, 1 = backward)
- Interrupt enable flag: Setting this enables interrupts
- Trace (trap) flag: Used for debugging. When set processor executes only 1 instruction, then interrupts to call debugger
- Sign flag: Set to the MSB of the result of an operation
- Zero flag: Set if the result is zero
- Auxiliary carry flag: Set if there was a carry from or borrow to bits 0-3 in the AL register
- Parity flag: Set if the number of1's in result is even
- Carry flag: Set if there was a carry from or borrow to the MSB during last calculation

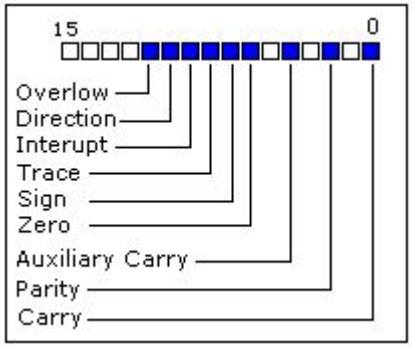


Image source: http://www.electronics.dit.ie/staff/tscarff/8086_registers/8086_registers.html

CMP

- CMP used to compare two 8, 16, or 32-bit numbers
- cmp operand1, operand2 => operand1-operand2
- The **cmp** instruction performs the same operation as the **sub** instruction except that the result of subtraction is **not saved**
- Thus, **cmp** does not disturb the source and destination operands
- After the subtraction operation, flags are set accordingly
- Flags register critical in branching

Branching and Loop Instructions

- JMP used to make the program execution jump to a specific label or address
  - Called an unconditional jump since jump always occurs
- jmp label
  - Label identifies the next instruction to be executed
- Example: Infinite loop

```
        mov ax, 1
inc_again:
        inc ax
        jmp inc_again
        mov bx, ax
```

Branching and Loop Instructions

- J<cond> used to make the program execution jump to a specific label or address **if the condition is true**
- j<cond> label
- Example

```
read_char:
        mov dl, 0
        …                   ;Code for reading a character into al
        cmp al, 0Dh         ;Compare the character to CR
        je CR_received      ;if equal, jump to CR_received
        inc cl              ;otherwise, increment cl and
        jmp read_char       ;go back to read another
CR_received:                ;character from keyboard
…
```

Branching and Loop Instructions

- Note, while the result is not saved anywhere, the operation sets the zero flag (ZF = 1) if the two operands are the same
- je jump if equal
- jg jump if greater
- jl jump if less
- jge jump if greater than or equal
- jle jump if less than or equal
- jne jump if not equal
- jz jump if zero
- jnz jump if not zero
- jc jump if carry
- jnc jump if not carry

Branching and Loop Instructions

- How is iteration performed?

```
        mov cl, 50          ;initialize loop counter
repeat1:
        <loop body>
        dec cl              ;update counter
        jnz repeat1         ;jumps back to
                            ;repeat1 if cl is not 0
```

Branching and Loop Instructions

- Alternatively use the loop instruction
- loop label
  - Uses cx register as loop counter
  - Decrements and then loops if cx ≠ 0

```
            mov cx, 50          ;initialize loop counter
repeat1:
            <loop body>
            loop repeat1
```

- Advantage: Auto-update counter,

Logical Instructions

- AND, OR, XOR, NOT
- and destination, source
- or destination, source
- xor destination, source
- not destination
- The first three are binary operators and perform bitwise and, or, and xor logical operations respectively.
- The not is a unary operator that performs bitwise complement operation
- Example:
  - and al, 01h
- Note: logical operations set the zero flag if the result is 0

Logical Instructions

- TEST performs logical bitwise **and** operation like the and instruction except that the source and destination are not modified
- However, test sets the flags just like the and instruction
- Example
  - test al, 01h

Shift Instructions

- SHL, SHR Shift logical left and shift logical right
- shl dest, count shr dest, count
- shl dest, cl shr dest, cl
- The destination can be an 8, 16, or 32-bit operand stored either in a register or in memory
- The second operand specifies the number of bit positions to be shifted
- The first format specifies the shift count directly. The shift count can range from 0 to 31
- The second format can be used to indirectly specify the shift count, which is assumed to be in the cl register. The cl register contents are not changed by either the shl or shr instructions

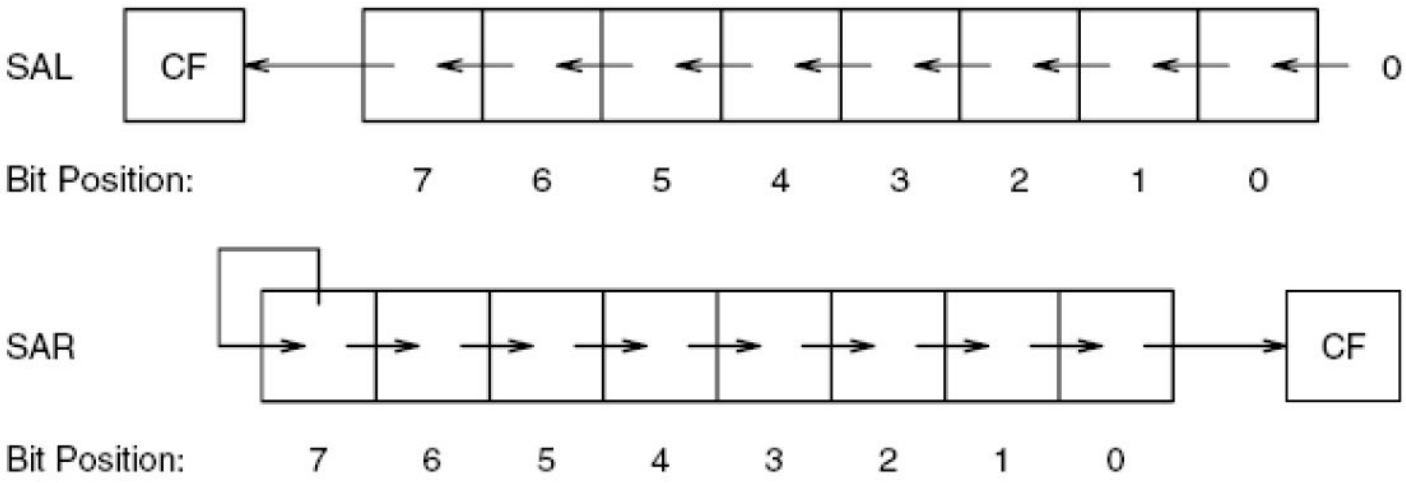
Shift Instructions

- Examples

	Before shift	After shift	
Instruction	AL or AX	AL or AX	CF
`shl AL,1`	1010 1110	0101 1100	1
`shr AL,1`	1010 1110	0101 0111	0
`mov CL,3` `shl AL,CL`	0110 1101	0110 1000	1
`mov CL,5` `shr AX,CL`	1011 1101 0101 1001	0000 0101 1110 1010	1

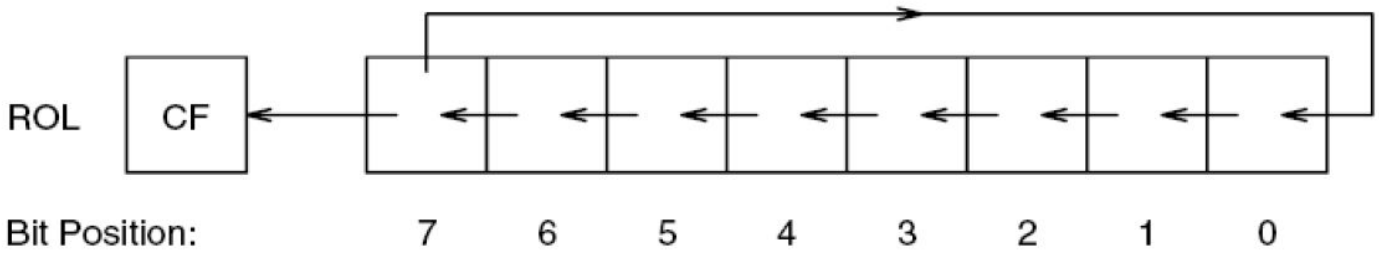
Shift Instructions

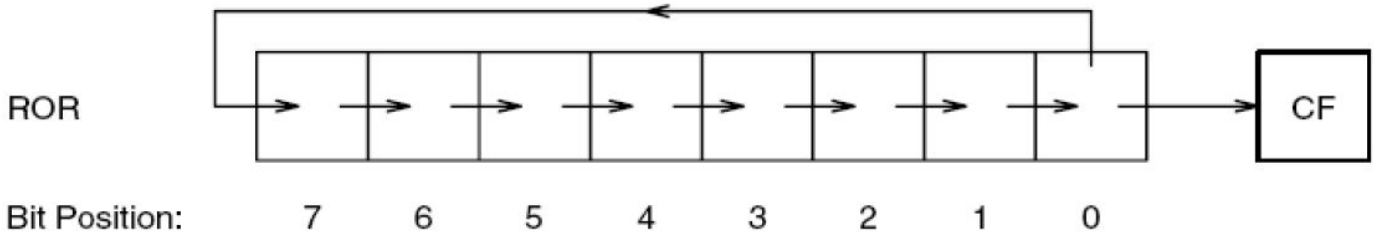
- SAL, SAR Shift arithmetic left and shift arithmetic right
- sal dest, count sar dest, count
- sal dest, cl sar dest, cl
- Examples



Rotate Instructions

- ROL, ROR Rotate left and right without carry
- rol dest, count ror dest, count
- rol dest, cl ror dest, cl





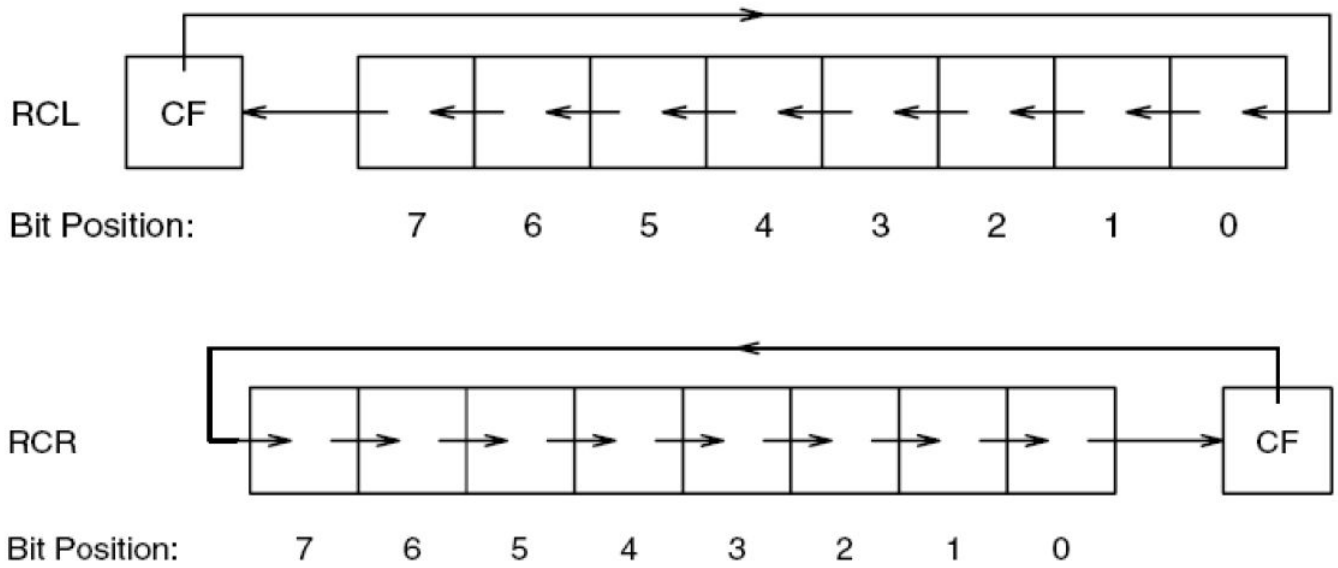
Rotate Instructions

- Examples

Instruction	Before execution	After execution	
	AL or AX	AL or AX	CF
`rol AL,1`	1010 1110	0101 1101	1
`ror AL,1`	1010 1110	0101 0111	0
`mov CL,3` `rol AL,CL`	0110 1101	0110 1011	1
`mov CL,5` `ror AX,CL`	1011 1101 0101 1001	1100 1101 1110 1010	1

Rotate Instructions

- RCL, RCR Rotate left and right through carry
- rcl dest, count rcr dest, count
- rcl dest, cl rcr dest, cl



Rotate Instructions

- Examples

Instruction	Before execution		After execution	
	AL or AX	CF	AL or AX	CF
`rcl AL,1`	1010 1110	0	0101 1100	1
`rcr AL,1`	1010 1110	1	1101 0111	0
`mov CL,3` `rcl AL,CL`	0110 1101	1	0110 1101	1
`mov CL,5` `rcr AX,CL`	1011 1101 0101 1001	0	1001 0101 1110 1010	1